8 API

9 Segmentation Methods

A concise overview of the segmentation algorithms available within the VisionToolkit library. For each method, a reference article that informed its implementation is cited.

9.1 Binary Segmentation

A `BinarySegmentation` object can be instantiated using the threshold-based methods included in this package:

- *I-VT*: Performs binary segmentation based on a velocity threshold [84].
- *I-DiT*: Performs binary segmentation using dispersion of consecutive data points within a temporal window — the dispersion metric is defined as the sum of the maximum and minimum differences between data coordinates [44].
- *I-DeT*: Performs binary segmentation using a density criteria [59]. This algorithm is a variant of the popular DBSCAN algorithm for fixation identification.
- *I-MST*: Performs binary segmentation leveraging minimum spanning trees. Building on this *tree representation*, I-MST algorithm classifies each eye position into a fixation or a saccade based on point to point edge distance thresholds [44].
- *I-KF*: Performs binary segmentation using a two state Kalman filter which models the evolution of the eyes by a dynamic system with two states: position and velocity [46].

A `BinarySegmentation` object can be instantiated using the learning-based segmentation methods included in our package:

- *I-HMM*: Performs binary segmentation assuming a two state — fixation and saccade — latent variable, where each state is characterized by a normal velocity distribution, while the switch between the different states is governed by a Markov jump process [84].
- *I-2MC*: Performs binary segmentation using a 2-means clustering approach [35].
- *I-RF*: Performs binary segmentation from a random forest classifier trained using a set of 14 features extracted from raw data eye-gaze coordinates [100]. This algorithm requires an annotated dataset for training.

9.2 Ternary Segmentation

A `TernarySegmentation` object can be instantiated using the threshold-based methods included in this package:

- *I-VVT*: Performs ternary segmentation using two velocity thresholds [45].
- *I-VDT*: Performs ternary segmentation using both a velocity and a dispersion thresholds [45].
- *I-VMP*: Performs ternary segmentation using eye movement patterns [2]: the angles between the horizontal axis and the line formed by two successive gaze-points are computed and then transformed into a set of points on the circumference of the unit circle. A Rayleigh score is then computed, and windows labeled consequently.
- *I-BDT*: Performs ternary segmentation using a Bayesian decision theory approach [86]. Building on physiological hypothesis, they introduced explicit formulas to compute likelihoods and priors of each eye movement that can be used to isolate event types.

A `TernarySegmentation` object can be instantiated using the learning-based segmentation methods included in our package:

- *I-HOV*: Performs ternary segmentation using from an histogram of oriented velocities [29]. The technique computes velocity-weighted angles between the inspected point and each preliminary or successive data-points to produce an histogram that can be used as features to feed machine learning algorithms. This algorithm requires an annotated dataset for training.
- *I-CNN*: Performs ternary segmentation using a convolutional neural network trained directly from raw eye-data in order to isolate oculomotor events [30]. This algorithm requires an annotated dataset for training.

10 Oculomotor Processing

A concise summary of the metrics included in the VisionToolkit library for characterizing detailed oculomotor behavior is provided. Each metric is accompanied by a reference to the source article that guided its implementation.

10.1 Signal-based Features

Frequency-based analysis:

- `signal_periodogram()`: Estimates power spectral density, using a periodogram [67], from raw eye-gaze signal.
- `signal_welch_periodogram()`: Estimates power spectral density, using a Welch windowed periodogram [96], from eye-gaze signal.
- `signal_cross_spectral_density()`: Estimates the cross power spectral density between two eye-gaze signals [67].
- `signal_welch_cross_spectral_density()`: Estimates the cross power spectral density between two eye-gaze signals, according to Welch's method [67].
- `signal_coherency()`: Derived from *cross spectral density*, assesses how two eye-gaze signals are related or the association between rythmic activities contained in both signals [7].

Stochastic-based analysis:

- `signal_MSD()`: A measure of the deviation of the eye-gaze position with respect to a reference position over time [34]. A useful tools that can be used to describe random walks.
- `signal_DACF()`: A representation of the degree of similarity between the eye-gaze coordinate time series and a lagged version of itself over successive time intervals [34].
- `signal_DFA()`: A method for determining the statistical self-affinity of a signal. It is useful for analysing time series that appear to be long-memory processes whose underlying statistics or dynamics are non-stationary [94].

Topological-based analysis:

- `signal_persistence_size()`: Computes the entropy of the *size* of the persistence diagram holes [17].
- `signal_persistence_robustness()`: Computes the entropy of the *robustness* of the holes in the persistence diagram [17].
- `signal_betti_curve()`: A one-dimensional function evaluating the Betti numbers obtained from a persistence diagram, at different levels of filtration [32].
- `signal_persistence_curve()`: A one dimensional function that summarizes the total persistence of topological hole obtained from a persistence diagram, at different levels of filtration [41].
- `signal_persistence_landscape()`: The persistence landscape is used to map persistence diagrams into a function space [32].

- `signal_persistence_entropy()`: The Shannon entropy of the collections of topological hole lifetimes obtained from a persistence diagram [41].

10.2 Fixation Features

Fixation temporal features:

- `fixation_frequency()`: Also referred to as *fixation rate* [82], this metric represents, given an eye-tracking recording session, the number of fixations occurring per second.
- `fixation_frequency_wrt_labels()`: Given an eye-tracking recording session, computes the number of fixations per second [82], filtered according to a quality criterion to retain only *validated* fixations.
- `fixation_durations()`: Given an eye-tracking recording session, computes the duration of each identified fixation sequence [82].
- `fixation_first_duration()`: Given an eye-tracking recording session, computes the duration of the first fixation sequence identified [40].

Fixation spatial and stability features:

- `fixation_centroids()`: Given an eye-tracking recording session, computes centroid position of each identified fixation sequence by averaging coordinates of data-samples composing individual fixations [82].
- `fixation_drift_displacements()`: Given an eye-tracking recording session, computes distance between the starting and ending points of each identified fixation sequence [82].
- `fixation_drift_distances()`: Given an eye-tracking recording session, for each identified fixation sequence, computes the sum of distances between each data sample within this fixation sequence [82].
- `fixation_mean_velocities()`: Given an eye-tracking recording session, for each identified fixation sequence, computes the mean velocity of data sample within this fixation sequence [82].
- `fixation_drift_velocities()`: Given an eye-tracking recording session, for each identified fixation sequence, computes the drift displacement normalized by the fixation duration [82].
- `fixation_BCEA()`: Given an eye-tracking recording session, for each identified fixation sequence, computes the fixation bivariate contour ellipse area (BCEA) as the area of the elliptical contour that encompasses a given percentage of sample points identified as belonging to this fixation sequence [88].

10.3 Saccade Features

Saccade temporal features:

- `saccade_frequency()`: Also referred to as *saccade rate* [82], this metric represents, given an eye-tracking recording session, the number of saccades occurring per second.
- `saccade_frequency_wrt_labels()`: Given an eye-tracking recording session, computes the number of saccades per second [82], filtered according to a quality criterion to retain only *validated* saccades.
- `saccade_durations()`: Given an eye-tracking recording session, computes the duration of each identified saccade sequence [82].

Saccade kinematic features:

- `saccade_amplitudes()`: Given an eye-tracking recording session, computes distance between the starting and ending points of each identified saccade sequence [82].

- `saccade_travel_distances()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the sum of distances between each data sample within this saccade sequence [82].
- `saccade_efficiencies()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the ratio of saccadic amplitude over the distance traveled [82].
- `saccade_directions()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the deviation from the horizontal plane of the line connecting the start and end points of the saccade sequence [27].
- `saccade_successive_deviations()`: Given an eye-tracking recording session, for each identified pair of successive saccade sequences, computes the angle formed by successive saccadic trajectories, where each saccade is modeled as a vector connecting its start and end points.
- `saccade_initial_directions()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the initial direction of the saccadic trajectory after a fixed number of data measures, *e.g.* after a number of times-stamps corresponding to 20 *ms* [90].
- `saccade_initial_deviations()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the angle between the overall direction determined at the endpoint of the saccade, and the initial direction after a fixed number of data measures, *e.g.* after a number of times-stamps corresponding to 10 *ms* [87].
- `saccade_max_curvatures()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the maximum perpendicular distance from any point along the saccadic trajectory to the straight line connecting the start and end points of the saccade [61].
- `saccade_area_curvatures()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the area under the curve of the sampled saccadic trajectory, relative to the straight-line distance between the saccade's starting and ending points [61].

Saccade velocity and acceleration features:

- `saccade_mean_velocities()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the mean velocity of data sample within this saccade sequence [82].
- `saccade_peak_velocities()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the peak velocity of data samples belonging to this saccade sequence [82].
- `saccade_acceleration_profiles()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the mean acceleration of data sample within this saccade sequence [82].
- `saccade_peak_accelerations()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the peak acceleration of data samples belonging to this saccade sequence [82].
- `saccade_accelerations()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the mean absolute acceleration during the acceleration phase of this saccade sequence, measured from the start point to the timestamp of peak acceleration [82].
- `saccade_decelerations()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the mean absolute acceleration during the deceleration phase of this saccade sequence, the timestamp of peak acceleration to the endpoint [82].

- `saccade_skewness_exponents()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the shape parameter obtained by fitting a gamma function to the saccade velocity profile [16].

Saccade functional ratios:

- `saccade_amp_dur_ratio()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the saccade amplitude over duration ratio [82].
- `saccade_peak_vel_amp_ratio()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the peak velocity over amplitude ratio [82].
- `saccade_peak_vel_dur_ratio()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the peak velocity over duration ratio [82].
- `saccade_peak_vel_vel_ratio()`: Given an eye-tracking recording session, for each identified saccade sequence, computes the peak velocity over mean velocity ratio [82].
- `saccade_main_sequence()`: Given an eye-tracking recording session, computes slopes of the amplitude / duration curve and the log peak velocity / log amplitude curve [6].

Saccade task features:

- `saccade_latencies()`: Given an eye-tracking recording session and a theoretical saccade sequence, computes the time difference between the onset of each theoretical saccade and the start time of each corresponding experimental saccade [98].
- `saccade_latency_quantiles()`: Given an eye-tracking recording session and a theoretical saccade sequence, computes the set of saccade latencies. Then computes quantiles of the latency distribution [91].
- `saccade_gain()`: Given an eye-tracking recording session and a theoretical saccade sequence, computes the ratio between saccade and target amplitudes — *i.e.* the distances covered by a theoretical saccade and the distance covered by the saccade actually performed [38].

10.4 Smooth Pursuit Features

Smooth pursuit temporal features:

- `pursuit_frequency()`: Also referred to as *pursuit rate*, this metric represents, given an eye-tracking recording session, the number of pursuit sequences occurring per second.
- `pursuit_frequency_wrt_labels()`: Given an eye-tracking recording session, computes the number of smooth pursuit per second, filtered according to a quality criterion to retain only *validated* smooth pursuit sequences.
- `pursuit_durations()`: Given an eye-tracking recording session, computes the duration of each identified smooth pursuit sequence.

Smooth pursuit kinematic features:

- `pursuit_amplitudes()`: Given an eye-tracking recording session, computes distance between the starting and ending points of each identified smooth pursuit sequence.
- `pursuit_travel_distances()`: Given an eye-tracking recording session, for each identified smooth pursuit sequence, computes the sum of distances between each data sample within this pursuit sequence
- `pursuit_efficiencies()`: Given an eye-tracking recording session, for each identified smooth pursuit sequence, computes the ratio of pursuit amplitude over the distance traveled

Smooth pursuit velocity features:

- `pursuit_mean_velocities()`: Given an eye-tracking recording session, for each identified smooth pursuit sequence, computes the mean velocity of data sample within this pursuit sequence.
- `pursuit_peak_velocities()`: Given an eye-tracking recording session, for each identified smooth pursuit sequence, computes the peak velocity of data samples belonging to this pursuit sequence

Smooth pursuit task features:

- `pursuit_latency()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes the time difference between the onset of each theoretical smooth pursuit and the start time of each corresponding experimental pursuit [13]. Computed as the abscissa of the intersection of two regression lines: he first line fits the pre-response baseline, and the second line fits the pursuit initiation velocity signal, recorded from a time window with duration of approximately 50 *ms*.
- `pursuit_initial_acceleration()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes the initial acceleration as the mean second-order position derivative of the saccade-free component extracted from the tracking response in the 100 *ms* interval immediately following pursuit onset [43].
- `pursuit_triangular_gain()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes the triangular gain as the ratio between eye and target mean velocities during each pursuit segment [79].
- `pursuit_sinusoidal_gain()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes the sinusoidal gain by fitting the eye velocity with a trigonometrical curve for each pursuit segment. Then gain is then computed as the ratio between the peak velocity of the best fitting curve over the target's peak velocity [1].
- `pursuit_sinusoidal_phase()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes the phase of the velocity signal as the difference between the phases of the best-fitting velocity curve and the target's velocity profile [1].
- `pursuit_accuracy()`: Estimates the proportion of time the smooth pursuit eye movement velocity is within the target velocity boundaries of less than 25% — adjustable — absolute error from the visual target velocity.
- `pursuit_error_entropy()`: Given an eye-tracking recording session and a theoretical pursuit sequence, first computes the pursuit velocity error series is first estimated as the difference between the experimental pursuit velocities and theoretical stimulus velocities. Then the approximate entropy [75] of the velocity error series is computed.
- `pursuit_cross_correlation()`: Given an eye-tracking recording session and a theoretical pursuit sequence, computes normalized cross-correlation [77] between the experimental pursuit velocity and theoretical stimulus velocity signals.

11 Scanpath Analysis

A concise summary of the metrics included in the VisionToolkit library for characterizing scanpath trajectory. Each metric is accompanied by a reference to the source article that guided its implementation.

11.1 Single Scanpath Analysis

Geometrical methods available:

- `scanpath_BCEA()`: Given a scanpath, computes the area of the elliptical contour that encompasses a given percentage of fixation centroids identified as belonging to this scanpath [88].
- `scanpath_k_coefficient()`: Given a scanpath, derived by averaging the differences, for each fixation, between the standardized *fixation duration* and the standardized *saccade amplitude* of the subsequent saccade composing this scanpath [48].
- `scanpath_convex_hull()`: Given a scanpath, computes the area of the smallest convex polygon that includes all fixations composing this scanpath [47].
- `scanpath_voronoi_cells()`: Given a scanpath, computes the skewness and scale parameter of a gamma distribution by fitting it to the distribution of normalized Voronoi cell areas [72]. A Voronoi cell is defined as the region containing all points in a plane that are closer to a specific fixation point than to any other fixation point within the scanpath.
- `scanpath_HFD()`: Given a scanpath, computes the Higuchi fractal dimension of Hilbert curve distance series obtained from fixation centroids belonging to this scanpath [70].

Saliency-based method available:

- `scanpath_saliency_map()`: Given a scanpath, or a list of scanpaths, computes a saliency-map or heat-map as an histogram where fixations from individual recordings, or aggregated across multiple observers, are counted at each pixel. Additionally, pixel activity is modeled as a Gaussian distribution, with the standard deviation determining the granularity of the heat map. The output is a map of relatively weighted pixels, where color or opacity reflects fixation density.

Recurrence quantification-based methods available:

- `scanapath_RQA_recurrence_rate()`: Given a recurrence plot computed from a single scanpath, derives the percentage of recurrent fixations, *i.e.* how often observers re-fixate previously fixated image positions [3].
- `scanapath_RQA_determinism()`: Given a recurrence plot computed from a single scanpath, derives the percentage of recurrence points that form diagonal lines in the recurrence plot [3].
- `scanapath_RQA_laminarity()`: Given a recurrence plot computed from a single scanpath, derives the percentage of entries from the upper half recurrence plot forming vertical and horizontal lines, which quantifies how often specific areas of a scene are repeatedly fixated [3].
- `scanapath_RQA_CORM()`: Given a recurrence plot computed from a single scanpath, derives the distance of the center of gravity of recurrent points from the diagonal of self-recurrence — the main diagonal of the recurrence plot [3].
- `scanapath_RQA_entropy()`: Given a recurrence plot computed from a single scanpath, derives the Shannon entropy of the distribution of diagonal line lengths, providing an estimate of the complexity within the scanpath's deterministic structure [53].

11.2 Scanpath Comparison Methods

Scanpath direct comparison methods:

- `scanpath_mannan_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the weighted mean distance between each fixation in one scanpath and its nearest neighbor — also known as *point-mapping* — in the other scanpath [62].
- `scanpath_eye_analysis_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the sum of all the point-mapping distances — from one sequence to the other and conversely — normalized by the number of points in the largest sequence [64].
- `scanpath_TDE_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the time delay embedding distance [93].
- `scanpath_DTW_distance()`: Given a list of scanpaths, this metric seeks, for each pair of scanpaths, the temporal alignment that minimizes Euclidean distance between aligned scanpath fixations [8].
- `scanpath_frechet_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the minimum of the maximum distances between two scanpaths when they are continuously aligned, preserving their sequential order [23].

The VisionToolkit library provides metrics for calculating scanpath string edit distances. Note that for the following metrics, scanpaths are first spatially — and, if desired, temporally — binarized to generate a sequence of characters.

- `scanpath_levenshtein_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the minimum number of single-character edits — insertions, deletions, or substitutions — needed to transform one sequence into another. Computed using the Wagner-Fischer algorithm [92].
- `scanpath_generalized_edit_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the minimum number of single-character edits — insertions, deletions, or substitutions — needed to transform one sequence into another. Also computed using the Wagner-Fischer algorithm [92], but allowing for distinct deletion and insertion costs, as well as a substitution cost based on the distance between spatial bins in the visual field.
- `scanpath_needleman_wunsch_distance()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the optimal alignment between two sequences through substitutions and gap insertions. While the Wagner-Fischer algorithm works with penalties for divergent segments the Needleman-Wunsch approach introduces matching bonuses between pairs of segments, allowing negative matches only when the segments are very different [69].

Scanpath saliency comparison methods between a reference saliency map and an observer scanpath:

- `scanpath_NSS()`: Given a scanpath and a *reference* saliency map, computes the *normalized scanpath saliency* metric by first normalizing the saliency map. This produces a z-score which shows how many standard deviations a particular fixation location is above chance. Finally, the NSS value for a given fixation location is computed on a small neighborhood centered on that location [].
- `scanpath_percentile()`: Given a scanpath and a *reference* saliency map, quantifies similarity by calculating the average percentile rank of each fixation's saliency value [74].
- `scanpath_ROC()`: Given a scanpath and a *reference* saliency map, leverages the *Area Under the ROC Curve* (AUC) measure to assess how well a reference saliency map predicts fixations by treating it as a binary classifier across varying thresholds [12].
- `scanpath_information_gain()`: Given a scanpath and a *reference* saliency map, measures the extent to which a saliency model predicts recorded fixations compared to a center prior baseline [52].

Scanpath saliency comparison methods between a pair of scanpaths:

- `scanpath_pearson()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the *Pearson correlation coefficient* as the linear relationship between two saliency map [56].
- `scanpath_KL()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the *Kullback-Leibler divergence* which measures the information loss when the input saliency map approximates the reference, with lower scores indicating a closer match [55].
- `scanpath_EMD()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, spatial differences between two distributions by measuring the minimum *cost* to morph one into the other [81].

Scanpath cross-recurrence quantification analysis:

- `scanapath_CRQA_recurrence_rate()`: Given a list of scanpaths, a cross-recurrence matrix is first computed from each pair of scanpath. From each matrix, cross recurrence rate is derived as the percentage of fixations that match between the two fixation sequences [63].
- `scanapath_CRQA_determinism()`: Given a list of scanpaths, a cross-recurrence matrix is first computed from each pair of scanpath. From each matrix, cross determinism is derived as the percentage of cross-recurrent points that form diagonal lines — representing fixation trajectories common to both fixation sequences [63].
- `scanapath_CRQA_laminarity()`: Given a list of scanpaths, a cross-recurrence matrix is first computed from each pair of scanpath. From each matrix, cross laminarity is derived as the percentage of consecutive recurrence points in one fixation series that are aligned vertically with recurrence points in the other series, forming vertical structures in the combined recurrence plot.[63].
- `scanapath_CRQA_entropy()`: Given a list of scanpaths, a cross-recurrence matrix is first computed from each pair of scanpath. From each matrix, cross entropy estimates the complexity of the attunement between the two scanpaths under study [63]. In other words, it estimates the complexity of the relation between eye-gaze data.

Scanpath specific similarity metrics:

- `scanpath_subsmatch_similarity()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the subsmatch similarity [50]. The method first converts scanpaths into character strings by spatially binning fixation sequences, then counting the number of occurrences of all sub-sequences of size n in each scanpath. Subsequently, the difference in occurrence frequency is used as the basis to evaluate similarity or dissimilarity between pairs of scanpaths.
- `scanpath_scanmatch_score()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the scanmatch score [18]. This method begins by converting scanpaths into character strings through spatial and temporal binning of fixation sequences. The resulting character sequences are then compared by maximizing a similarity score computed using the Needleman-Wunsch algorithm. The core of the ScanMatch approach, however, is in its construction of a similarity matrix for regions within the visual field: a threshold is applied, making substitution matrix values positive for closely related regions and negative for regions with weaker relationships.
- `scanpath_multimatch_alignment()`: Given a list of scanpaths, this metric computes, for each pair of scanpaths, the multimatch alignment algorithm [21], a vector-based, multi-dimensional method for assessing scanpath similarity. In this approach, vectorized and simplified scanpaths are sss, allowing for a comprehensive multidimensional similarity assessment.

12 Areas of Interest

A concise summary of the metrics and methods included in the VisionToolkit library for characterizing AoI sequences. Each metric is accompanied by a reference to the source article that guided its implementation.

12.1 AoI Identification

AoI identification methods available:

- *I_AP*: Performs AoI identification using an affinity propagation algorithm which identifies exemplars — representative fixation points for each AoI — by iteratively passing messages between data points based on their similarity [28].
- *I_DP*: Performs AoI identification using density peak clustering, which considers two properties of fixation points, namely local density — computed as the number of fixation points within a user-defined distance — and distance from points with higher density [57].
- *I_DT*: Performs AoI identification using the DBSCAN clustering method, which detects areas of high density separated by low-density regions [26].
- *I_KM*: Performs AoI identification using a K-means clustering method [76].
- *I_MS*: Performs AoI identification using mean shift clustering which iteratively shifts data points toward the densest region in feature space, then apply a distance threshold to separate them into clusters [85].

12.2 Markov-Based Analysis

AoI Markov-based analysis:

- `AoI_transition_matrix()`: Given an AoI sequence, computes transition probabilities between visual elements, represented as a transition matrix [39].
- `AoI_transition_entropy()`: Given an AoI sequence, computes transition probabilities between visual elements, represented as a transition matrix. Then, from the inferred transition matrix, the following entropies are computed: stationary entropy, joint entropy, conditional entropy and mutual information [49].
- `AoI_HMM()`:Given a fixation sequence, learns parameters of an Hidden Markov Model using a Baum-Welch algorithm [9].
- `AoI_HMM_transition_entropy()`: Given a fixation sequence, learns parameters of an Hidden Markov Model using a Baum-Welch algorithm. Then, from the inferred transition matrix, the following entropies are computed: stationary entropy, joint entropy, conditional entropy and mutual information [49].
- `AoI_HMM_fisher_vector()`: Given a fixation sequence, learns parameters of an Hidden Markov Model and the corresponding Fisher vector [42]. The idea of a Fisher kernel is to compute how a new sequence of gaze data would change the parameters of the model — *i.e.* the parameter gradients of the generative model when given a novel time-series as input.

12.3 AoI Pattern Mining

Pattern mining methods available:

- `AoI_lempel_ziv()`: Given an AoI sequence, quantifies the complexity of visual scanning paths [60]. The AoI sequence is scanned from left to right in order to identify new elements or sets of elements which have not yet

been identified, then stored in a code-book. The complexity of Lempel-Ziv is a numerical value computed by considering the number of different patterns recorded in the code book.

- `AoI_ngram()`: Given an AoI sequence, assesses the distribution of fixed-length subsequences within gaze data. The possible n-grams with a given alphabet are calculated using combinatorics, and their occurrence in the AoI sequence counted. Subsequently, their frequencies and distributions are derived [51].
- `AoI_t_patterns()`: Given an AoI sequence, identify statistically recurrent temporal patterns in visual behaviors, with tolerance for gaps [83].
- `AoI_SPAM()`: Given a list of AoI sequences, computes the Sequential Pattern Mining algorithm which finds frequent sequences or patterns in a dataset. It combines depth-first search with efficient bitwise operations to track sequence occurrences [5].

12.4 AoI Sequence Global Alignment

Alignment methods available:

- `AoI_levenshtein_distance()`: Given a list of AoI sequences, this metric computes, for each pair of AoI sequences, the minimum number of single-character edits — insertions, deletions, or substitutions — needed to transform one AoI sequence into another. Computed using the Wagner-Fischer algorithm [92].
- `AoI_generalized_edit_distance()`: Given a list of AoI sequences, this metric computes, for each pair of AoI sequences, the minimum number of single-character edits — insertions, deletions, or substitutions — needed to transform one AoI sequence into another. Also computed using the Wagner-Fischer algorithm [92], but allowing for distinct deletion and insertion costs, as well as a substitution cost based on the distance between spatial bins in the visual field.
- `AoI_needleman_wunsch_distance()`: Given a list of AoI sequences, this metric computes, for each pair of AoI sequences, the optimal alignment between two AoI sequences through substitutions and gap insertions. While the Wagner-Fischer algorithm works with penalties for divergent segments the Needleman-Wunsch approach introduces matching bonuses between pairs of segments, allowing negative matches only when the segments are very different [69].

12.5 AoI Common Subsequence

Available methods to assess common subsequences:

- `AoI_longest_common_subsequence()`: Given a list of AoI sequences, this metric identifies, for each pair of AoI sequences, the longest subsequence common to two sequences by using dynamic programming.
- `AoI_smith_waterman()`: Given a list of AoI sequences, this metric identifies, for each pair of AoI sequences, similar patterns among observer AoI sequences using a dynamic programming approach that incorporates a gap penalty function and a substitution matrix. This matrix is based on the distances between AOIs and employs a threshold value for scoring matches and mismatches [97].
- `AoI_eMine()`: Given a list of AoI sequences, it firstly chooses the two most similar sequences according to the Levenshtein distance, before applying the longest common subsequence technique to these two scanpaths to find their common scanpath. Subsequently, the two selected sequences are removed from the input set and replaced by their longest common sequence. The process is repeated until there is a single sequence remaining, providing a common AoI sequence [24].

- `AoI_trend_analysis()`: Given a list of AoI sequences, the scanpath trend analysis algorithm first analyzes the most-visited visual elements, groups the sub-sequences by organizing these visual elements according to their overall position in the individual sequences, and finally builds a trend sequence based on these visual elements [25].
- `AoI_CDBA()`: Given a list of AoI sequences, the candidate-constrained dynamic time warping barycenter averaging iteratively aggregates multiple sequences into a single one by computing the barycenter of the experimentally recorded sequences [58].
- `AoI_hierarchical_clustering()`: Given a list of AoI sequences, produces the hierarchical clustering from the distance matrix provided by the Smith-Waterman pair distances between input sequences.[97]
- `AoI_dotplot_clustering()`: Given a list of AoI sequences, produces the hierarchical clustering from the distance matrix computed through a two steps procedure [31].

12.6 AoI Visualization Tools

- `AoI_time_plot()`: Given an AoI sequence, return a time plot which displays discrete AoIs on the vertical axis, while time is indicated on the vertical axis [78].
- `AoI_scarf_plot()`: Given a list of AoI sequences, plots a scarf diagram — a visualization tool designed to compare AoI sequences across different observers. The diagram represents various areas of interest along the vertical axis and time progression along the horizontal axis. Each AoI is assigned a unique color, with a separate time bar displayed for each observer, facilitating visual comparison of their gaze patterns over time [10].
- `AoI_sankey_diagram()`: Given a list of AoI sequences, plots a Sankey diagram, a convenient visualization method for investigating time varying fixation frequencies together with transitions between areas of interest obtained from a large number of inout AoI sequences [10].
- `AoI_directed_graph()`: Given an AoI sequence and its transition matrix, plots a a directed graph which shows the transitions between AoIs, with each node representing an area of interest, while links describe the transitions [10].
- `AoI_chord_diagram()`: Given an AoI sequence and its transition matrix, plots a chord diagram, a popular high-dimensional visualization tool for representing connections between nodes [95]. Their simplicity and intuitive design — a radial layout allows all nodes to be directly connected to all other nodes — results in common use for affiliation visualization
- `AoI_hierarchical_flow()`: Given an AoI sequence and its transition matrix, plots a flow diagram where the different AoIs represent vertices of a graph and are visualized as rectangular boxes. These boxes are organized in vertically stacked layers, depending on their average order in AoI sequence [11].

Acknowledgments

The authors declare no additional sources of funding

References

[1] AP Accardo, S Pensiero, S Da Pozzo, and P Perissutti. 1995. Characteristics of horizontal smooth pursuit eye movements to sinusoidal stimulation in children of primary school age. *Vision research* 35, 4 (1995), 539–548.

[2] JS Agustin. 2009. Off-the-shelf gaze interaction. *PhD PhD Thesis, The IT University of Copenhagen* (2009).

[3] Nicola C Anderson, Walter F Bischof, Kaitlin EW Laidlaw, Evan F Risko, and Alan Kingstone. 2013. Recurrence quantification analysis of eye movements. *Behavior research methods* 45 (2013), 842–856.

[4] Javier Andreu-Perez, Celine Solnais, and Kumuthan Sriskandarajah. 2016. EALab (Eye Activity Lab): a MATLAB Toolbox for variable extraction, multivariate analysis and classification of eye-movement data. *Neuroinformatics* 14 (2016), 51–67.

[5] Jay Ayres, Jason Flannick, Johannes Gehrke, and Tomi Yiu. 2002. Sequential pattern mining using a bitmap representation. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 429–435.

[6] A Terry Bahill, Michael R Clark, and Lawrence Stark. 1975. The main sequence, a tool for studying human eye movements. *Mathematical biosciences* 24, 3-4 (1975), 191–204.

[7] JS Bendat and AG Piersol. 1986. Random data: Analysis and measurement procedures 2nd Edition A Wiley-Interscience Publication. *New York* (1986).

[8] Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*. 359–370.

[9] Jeff A Bilmes et al. 1998. A gentle tutorial of the EM algorithm and its application to parameter estimation for Gaussian mixture and hidden Markov models. *International computer science institute* 4, 510 (1998), 126.

[10] Tanja Blascheck, Kuno Kurzhals, Michael Raschke, Michael Burch, Daniel Weiskopf, and Thomas Ertl. 2017. Visualization of eye tracking data: A taxonomy and survey. In *Computer Graphics Forum*, Vol. 36. Wiley Online Library, 260–284.

[11] Michael Burch, Ayush Kumar, and Klaus Mueller. 2018. The hierarchical flow of eye movements. In *Proceedings of the 3rd Workshop on Eye Tracking and Visualization*. 1–5.

[12] Zoya Bylinskii, Tilke Judd, Aude Oliva, Antonio Torralba, and Frédo Durand. 2018. What do different evaluation metrics tell us about saliency models? *IEEE transactions on pattern analysis and machine intelligence* 41, 3 (2018), 740–757.

[13] JR Carl and RS Gellman. 1987. Human smooth pursuit: stimulus-dependent responses. *Journal of Neurophysiology* 57, 5 (1987), 1446–1463.

[14] Benjamin T Carter and Steven G Luke. 2020. Best practices in eye tracking research. *International Journal of Psychophysiology* 155 (2020), 49–62.

[15] Laura Cercenelli, Guido Tiberi, Ivan Corazza, Giuseppe Giannaccare, Michela Fresina, and Emanuela Marcelli. 2017. SacLab: A toolbox for saccade analysis to increase usability of eye tracking systems in clinical ophthalmology practice. *Computers in Biology and Medicine* 80 (2017), 45–55.

[16] YUNG-FU CHEN, HSUAN-HUNG LIN, TAINSONG CHEN, TZU-TUNG TSAI, and I-FEN SHEE. 2002. The peak velocity and skewness relationship for the reflexive saccades. *Biomedical Engineering: Applications, Basis and Communications* 14, 02 (2002), 71–80.

[17] Yu-Min Chung, Chuan-Shen Hu, Yu-Lun Lo, and Hau-Tieng Wu. 2021. A persistent homology approach to heart rate variability analysis with an application to sleep-wake classification. *Frontiers in physiology* 12 (2021), 637684.

[18] Filipe Cristino, Sebastiaan Mathôt, Jan Theeuwes, and Iain D Gilchrist. 2010. ScanMatch: A novel method for comparing fixation sequences. *Behavior research methods* 42, 3 (2010), 692–700.

[19] Edwin S Dalmaijer, Sebastiaan Mathôt, and Stefan Van der Stigchel. 2014. PyGaze: An open-source, cross-platform toolbox for minimal-effort programming of eyetracking experiments. *Behavior research methods* 46 (2014), 913–921.

[20] Théo De La Hogue, Damien Mouratille, Mickaël Causse, and Jean-Paul Imbert. 2024. ArGaze: An Open and Flexible Software Library for Gaze Analysis and Interaction. (2024).

[21] Richard Dewhurst, Marcus Nyström, Halszka Jarodzka, Tom Foulsham, Roger Johansson, and Kenneth Holmqvist. 2012. It depends on how you look at it: Scanpath comparison in multiple dimensions with MultiMatch, a vector-based approach. *Behavior research methods* 44 (2012), 1079–1100.

[22] Andrew T Duchowski and Andrew T Duchowski. 2017. The gaze analytics pipeline. *Eye Tracking Methodology: Theory and Practice* (2017), 175–191.

[23] Thomas Eiter and Heikki Mannila. 1994. Computing discrete Fréchet distance. (1994).

[24] Sukru Eraslan, Yeliz Yesilada, and Simon Harper. 2014. Identifying patterns in eyetracking scanpaths in terms of visual elements of web pages. In *Web Engineering: 14th International Conference, ICWE 2014, Toulouse, France, July 1-4, 2014. Proceedings 14*. Springer, 163–180.

[25] Sukru Eraslan, Yeliz Yesilada, and Simon Harper. 2016. Scanpath trend analysis on web pages: Clustering eye tracking scanpaths. *ACM Transactions on the Web (TWEB)* 10, 4 (2016), 1–35.

[26] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.

[27] Tom Foulsham, Alan Kingstone, and Geoffrey Underwood. 2008. Turning the world around: Patterns in saccade direction vary with picture orientation. *Vision research* 48, 17 (2008), 1777–1790.

[28] Brendan J Frey and Delbert Dueck. 2007. Clustering by passing messages between data points. *science* 315, 5814 (2007), 972–976.

[29] Wolfgang Fuhl, Nora Castner, and Enkelejda Kasneci. 2018. Histogram of oriented velocities for eye movement detection. In *Proceedings of the Workshop on Modeling Cognitive Processes from Multimodal Data*. 1–6.

[30] Wolfgang Fuhl, Yao Rong, and Enkelejda Kasneci. 2021. Fully convolutional neural networks for raw eye tracking data segmentation, generation, and reconstruction. In *2020 25th International Conference on Pattern Recognition (ICPR)*. IEEE, 142–149.

[31] Joseph H Goldberg and Jonathan I Helfman. 2010. Scanpath clustering and aggregation. In *Proceedings of the 2010 symposium on eye-tracking research & applications*. 227–234.

[32] İsmail Güzel and Atabey Kaygun. 2023. Classification of stochastic processes with topological data analysis. *Concurrency and Computation: Practice and Experience* 35, 24 (2023), e7732.

[33] Mary Hayhoe and Dana Ballard. 2005. Eye movements in natural behavior. *Trends in cognitive sciences* 9, 4 (2005), 188–194.

[34] Carl JJ Herrmann, Ralf Metzler, and Ralf Engbert. 2017. A self-avoiding walk with neural delays as a model of fixational eye movements. *Scientific reports* 7, 1 (2017), 12958.

[35] Roy S Hessels, Diederick C Niehorster, Chantal Kemner, and Ignace TC Hooge. 2017. Noise-robust fixation detection in eye movement data: Identification by two-means clustering (I2MC). *Behavior research methods* 49, 5 (2017), 1802–1823.

[36] Roy S Hessels, Diederick C Niehorster, Marcus Nyström, Richard Andersson, and Ignace TC Hooge. 2018. Is the eye-movement field confused about fixations and saccades? A survey among 124 researchers. *Royal Society open science* 5, 8 (2018), 180502.

[37] James E Hoffman. 2016. Visual attention and eye movements. *Attention* (2016), 119–153.

[38] K. Holmqvist, M. Nystrom, R. Andersson, R. Dewhurst, H. Jarodzka, and J. Van de Weijer. 2011. Eye tracking: A comprehensive guide to methods and measures. *OUP Oxford* (2011).

[39] Janet H Hsiao, Hui Lan, Yueyuan Zheng, and Antoni B Chan. 2021. Eye movement analysis with hidden Markov models (EMHMM) with co-clustering. *Behavior Research Methods* 53, 6 (2021), 2473–2486.

[40] Albrecht W Inhoff, Ralph Radach, Matt Starr, and Seth Greenberg. 2000. Allocation of visuo-spatial attention and saccade programming during reading. In *Reading as a perceptual process*. Elsevier, 221–246.

[41] Oleg Kachan and Arsenii Onuchin. 2021. Topological data analysis of eye movements. In *2021 IEEE 18th International Symposium on Biomedical Imaging (ISBI)*. IEEE, 1398–1401.

[42] Christopher Kanan, Nicholas A Ray, Dina NF Bseiso, Janet H Hsiao, and Garrison W Cottrell. 2014. Predicting an observer's task using multi-fixation pattern analysis. In *Proceedings of the symposium on eye tracking research and applications*. 287–290.

[43] Grace W Kao and Mark J Morrow. 1994. The relationship of anticipatory smooth eye movement to smooth pursuit initiation. *Vision research* 34, 22 (1994), 3027–3036.

[44] Oleg V Komogortsev, Denise V Gobert, Sampath Jayarathna, Sandeep M Gowda, et al. 2010. Standardization of automated analyses of oculomotor fixation and saccadic behaviors. *IEEE Transactions on biomedical engineering* 57, 11 (2010), 2635–2645.

[45] Oleg V Komogortsev and Alex Karpov. 2013. Automated classification and scoring of smooth pursuit eye movements in the presence of fixations and saccades. *Behavior research methods* 45, 1 (2013), 203–215.

[46] Oleg V Komogortsev and Javed I Khan. 2007. Kalman filtering in the design of eye-gaze-guided computer interfaces. In *International Conference on Human-Computer Interaction*. Springer, 679–689.

[47] Xerxes P Kotval and Joseph H Goldberg. 1998. Eye movements and interface component grouping: An evaluation method. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 42. SAGE Publications Sage CA: Los Angeles, CA, 486–490.

[48] Krzysztof Krejtz, Andrew Duchowski, Izabela Krejtz, Agnieszka Szarkowska, and Agata Kopacz. 2016. Discerning ambient/focal attention with coefficient K. *ACM Transactions on Applied Perception (TAP)* 13, 3 (2016), 1–20.

[49] Krzysztof Krejtz, Andrew Duchowski, Tomasz Szmidt, Izabela Krejtz, Fernando González Perilli, Ana Pires, Anna Vilaro, and Natalia Villalobos. 2015. Gaze transition entropy. *ACM Transactions on Applied Perception (TAP)* 13, 1 (2015), 1–20.

[50] Thomas C Kübler, Enkelejda Kasneci, and Wolfgang Rosenstiel. 2014. Subsmatch: Scanpath similarity in dynamic scenes based on subsequence frequencies. In *Proceedings of the Symposium on Eye Tracking Research and Applications*. 319–322.

[51] Thomas C Kübler, Colleen Rothe, Ulrich Schiefer, Wolfgang Rosenstiel, and Enkelejda Kasneci. 2017. SubsMatch 2.0: Scanpath comparison and classification based on subsequence frequencies. *Behavior research methods* 49, 3 (2017), 1048–1064.

[52] Matthias Kümmerer, Thomas Wallis, and Matthias Bethge. 2014. How close are we to understanding image-based saliency? *arXiv preprint arXiv:1409.7686* (2014).

[53] Antonio Lanata, Laura Sebastiani, Francesco Di Gruttola, Stefano Di Modica, Enzo Pasquale Scilingo, and Alberto Greco. 2020. Nonlinear analysis of eye-tracking information for motor imagery assessments. *Frontiers in Neuroscience* 13 (2020), 1431.

[54] Michael F Land. 2009. Vision, eye movements, and natural behavior. *Visual neuroscience* 26, 1 (2009), 51–62.

[55] Olivier Le Meur, Patrick Le Callet, and Dominique Barba. 2007. Predicting visual fixations on video based on low-level visual features. *Vision research* 47, 19 (2007), 2483–2498.

[56] Olivier Le Meur, Patrick Le Callet, Dominique Barba, and Dominique Thoreau. 2006. A coherent computational approach to model bottom-up visual attention. *IEEE transactions on pattern analysis and machine intelligence* 28, 5 (2006), 802–817.

[57] Aoqi Li and Zhenzhong Chen. 2018. Representative scanpath identification for group viewing pattern analysis. *journal of eye movement research* 11, 6 (2018).

[58] Aoqi Li, Yingxue Zhang, and Zhenzhong Chen. 2017. Scanpath mining of eye movement trajectories for visual attention analysis. In *2017 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 535–540.

[59] Beibin Li, Quan Wang, Erin Barney, Logan Hart, Carla Wall, Katarzyna Chawarska, Irati Saez de Urabain, Timothy J. Smith, and Frederick Shic. 2016. Modified DBSCAN Algorithm on Oculomotor Fixation Identification. In *Proceedings of the Ninth Biennial ACM Symposium on Eye Tracking Research and Applications* (Charleston, South Carolina) *(ETRA '16)*. Association for Computing Machinery, New York, NY, USA, 337–338. https://doi.org/10.1145/2857491.2888587

[60] C. Lounis, V. Peysakhovich, and M Causse. 2020. Complexity of dwell sequences: visual scanning pattern differences between novice and expert aircraft pilots. *In 1st International Workshop on Eye-Tracking in Aviation* (2020).

[61] Casimir JH Ludwig and Iain D Gilchrist. 2002. Measuring saccade curvature: A curve-fitting approach. *Behavior Research Methods, Instruments, & Computers* 34 (2002), 618–624.

[62] S Mannan, Keith H Ruddock, and David S Wooding. 1995. Automatic control of saccadic eye movements made in visual inspection of briefly presented 2-D images. *Spatial vision* 9, 3 (1995), 363–386.

[63] Norbert Marwan, M Carmen Romano, Marco Thiel, and Jürgen Kurths. 2007. Recurrence plots for the analysis of complex systems. *Physics reports* 438, 5-6 (2007), 237–329.

[64] Sebastiaan Mathôt, Filipe Cristino, Iain D Gilchrist, and Jan Theeuwes. 2012. A simple way to estimate similarity between pairs of eye movement sequences. *Journal of Eye Movement Research* 5, 1 (2012), 1–15.

[65] James G May, Robert S Kennedy, Mary C Williams, William P Dunlap, and Julie R Brannan. 1990. Eye movement indices of mental workload. *Acta psychologica* 75, 1 (1990), 75–89.

[66] Michael B McCamy, Jorge Otero-Millan, Leandro Luigi Di Stasi, Stephen L Macknik, and Susana Martinez-Conde. 2014. Highly informative natural scene regions increase microsaccade production during visual scanning. *Journal of neuroscience* 34, 8 (2014), 2956–2966.

[67] Clare D McGillem and George R Cooper. 1991. Continuous and discrete signal and system analysis. *(No Title)* (1991).

[68] Kristin Moore and Leo Gugerty. 2010. Development of a novel measure of situation awareness: The case for eye movement analysis. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 54. Sage Publications Sage CA: Los Angeles, CA, 1650–1654.

[69] Saul B Needleman and Christian D Wunsch. 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of molecular biology* 48, 3 (1970), 443–453.

[70] Robert Ahadizad Newport, Carlo Russo, Abdulla Al Suman, and Antonio Di Ieva. 2021. Assessment of eye-tracking scanpath outliers using fractal geometry. *Heliyon* 7, 7 (2021).

[71] Jorge Otero-Millan, Xoana G Troncoso, Stephen L Macknik, Ignacio Serrano-Pedraza, and Susana Martinez-Conde. 2008. Saccades and microsaccades during visual fixation, exploration, and search: foundations for a common saccadic generator. *Journal of vision* 8, 14 (2008), 21–21.

[72] Eelco AB Over, Ignace TC Hooge, and Casper J Erkelens. 2006. A quantitative measure for the uniformity of fixation density: The Voronoi method. *Behavior research methods* 38 (2006), 251–261.

[73] Rémi Pautrat, Iago Suárez, Yifan Yu, Marc Pollefeys, and Viktor Larsson. 2023. Gluestick: Robust image matching by sticking points and lines together. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 9706–9716.

[74] Robert J Peters and Laurent Itti. 2008. Applying computational tools to predict gaze direction in interactive visual environments. *ACM Transactions on Applied Perception (TAP)* 5, 2 (2008), 1–19.

[75] S.M. Pincus, I.M. Gladstone, and R.A. Ehrenkranz. 1991. A regularity statistic for medical data analysis. *Journal of Clinical Monitoring and Computing* 7 (1991), 335–345.

[76] Claudio M. Privitera and Lawrence W. Stark. 2000. Algorithms for defining visual regions-of-interest: Comparison with eye fixations. *IEEE Transactions on pattern analysis and machine intelligence* 22, 9 (2000), 970–982.

[77] Lawrence R Rabiner. 1978. *Digital processing of speech signals*. Pearson Education India.

[78] Kari-Jouko Räihä, Anne Aula, Päivi Majaranta, Harri Rantala, and Kimmo Koivunen. 2005. Static visualization of temporal eye-tracking data. In *Human-Computer Interaction-INTERACT 2005: IFIP TC13 International Conference, Rome, Italy, September 12-16, 2005. Proceedings 10*. Springer, 946–949.

[79] Cyril Rashbass. 1961. The relationship between saccadic and smooth tracking eye movements. *The Journal of physiology* 159, 2 (1961), 326.

[80] Erik D Reichle, Alexander Pollatsek, Donald L Fisher, and Keith Rayner. 1998. Toward a model of eye movement control in reading. *Psychological review* 105, 1 (1998), 125.

[81] Nicolas Riche, Matthieu Duvinage, Matei Mancas, Bernard Gosselin, and Thierry Dutoit. 2013. Saliency and human fixations: State-of-the-art and study of comparison metrics. In *Proceedings of the IEEE international conference on computer vision*. 1153–1160.

[82] Ioannis Rigas, Lee Friedman, and Oleg Komogortsev. 2018. Study of an extensive set of eye movement features: Extraction methods and statistical analysis. *Journal of Eye Movement Research* 11, 1 (2018).

[83] Albert Ali Salah, Eric Pauwels, Romain Tavenard, and Theo Gevers. 2010. T-patterns revisited: mining for temporal patterns in sensor data. *Sensors* 10, 8 (2010), 7496–7513.

[84] Dario D Salvucci and Joseph H Goldberg. 2000. Identifying fixations and saccades in eye-tracking protocols. In *Proceedings of the 2000 symposium on Eye tracking research & applications*. 71–78.

[85] Anthony Santella and Doug DeCarlo. 2004. Robust clustering of eye movement recordings for quantification of visual interest. In *Proceedings of the 2004 symposium on Eye tracking research & applications*. 27–34.

[86] Thiago Santini, Wolfgang Fuhl, Thomas Kübler, and Enkelejda Kasneci. 2016. Bayesian identification of fixations, saccades, and smooth pursuits. In *Proceedings of the Ninth Biennial ACM Symposium on Eye Tracking Research & Applications*. 163–170.

[87] Boris M Sheliga, Lucia Riggio, Laila Craighero, and Giacomo Rizzolatti. 1995. Spatial attention-determined modifications in saccade trajectories. *Neuroreport* 6, 3 (1995), 585.

[88] Robert M Steinman. 1965. Effect of target size, luminance, and color on monocular fixation. *JOSA* 55, 9 (1965), 1158–1164.

[89] Jan Theeuwes, Artem Belopolsky, and Christian NL Olivers. 2009. Interactions between working memory, attention and eye movements. *Acta psychologica* 132, 2 (2009), 106–114.

[90] JAM Van Gisbergen, AJ Van Opstal, and JGH Roebroek. 1987. Stimulus-induced midflight modification of saccade trajectories. In *Eye movements from physiology to cognition*. Elsevier, 27–36.

[91] Cecile Vullings. 2018. *Saccadic latencies depend on functional relations with the environment*. Ph. D. Dissertation. Universite de Lille.

[92] Robert A Wagner and Michael J Fischer. 1974. The string-to-string correction problem. *Journal of the ACM (JACM)* 21, 1 (1974), 168–173.

[93] Wei Wang, Cheng Chen, Yizhou Wang, Tingting Jiang, Fang Fang, and Yuan Yao. 2011. Simulating human saccadic scanpaths on natural images. In *CVPR 2011*. IEEE, 441–448.

[94] Yanjun Wang, Wei Cong, Bin Dong, Fan Wu, and Minghua Hu. 2015. Statistical analysis of air traffic controllers' eye movements. In *The 11th USA/Europe ATM R&D Seminar*, Vol. 2015. 9.

[95] Zhiguo Wang. 2021. Eye Movement Data Analysis and Visualization. In *Eye-Tracking with Python and Pylink*. Springer, 197–224.

[96] P. Welch. 1967. The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms. *IEEE Transactions on Audio and Electroacoustics* 15 (1967), 70–73.

[97] Julia M West, Anne R Haake, Evelyn P Rozanski, and Keith S Karn. 2006. eyePatterns: software for identifying patterns and similarities across fixation sequences. In *Proceedings of the 2006 symposium on Eye tracking research & applications*. 149–154.

[98] Robert Whelan. 2008. Effective analysis of reaction time data. *The psychological record* 58, 3 (2008), 475–482.

[99] Jian Wu, Zhiming Cui, Victor S Sheng, Pengpeng Zhao, Dongliang Su, and Shengrong Gong. 2013. A Comparative Study of SIFT and its Variants. *Measurement science review* 13, 3 (2013), 122–131.

[100] Raimondas Zemblys, Diederick C Niehorster, Oleg Komogortsev, and Kenneth Holmqvist. 2018. Using machine learning to detect events in eye-tracking data. *Behavior research methods* 50 (2018), 160–181.

Received 1 November 2024; revised ; accepted